

Natural Language Access to Domain-Specific Metadata: A Reusable Framework for LLM Query Generation

Blake G. Fitch

blake.fitch@tuebingen.mpg.de

Max Planck Institute for Biological Cybernetics
Tübingen, Germany

Cato Elia Kurtz

Max Planck Institute for Biological Cybernetics
Tübingen, Germany

Abstract

Researchers need to answer ad-hoc questions about the contents of domain-specific archives but often lack the expertise to write structured queries on the metadata. We show that when domain vocabulary and semantics are captured in a well-designed Web Ontology Language (OWL) ontology, Large Language Models (LLMs) can generate accurate structured queries zero-shot, without fine-tuning, retrieval augmentation, or multi-agent orchestration. We present the Natural Language Knowledge Graph Query (NLKGQ) system, a framework and development process that enables natural language access to metadata in such archives. The framework includes a web interface that helps researchers pose natural language questions, which a domain-agnostic harness translates to SPARQL via an LLM and executes against a knowledge graph. The development process begins with capturing domain vocabulary and semantics in a formal OWL ontology. Domain-specific code then extracts metadata from archive sources and imports it into a knowledge graph defined by the ontology. Both the framework and process are designed to support reuse across multiple domains. In this work, we demonstrate the system for metadata derived from a large-scale neuroimaging research archive, evaluating performance across multiple LLMs and ontology representations. The best configurations achieve 100% accuracy on a competence and regression question set developed with domain experts. An ablation study across eight ontology representations reveals that readable entity names and semantic annotations are the dominant factors in accuracy, more significant than model choice or prompt engineering. We also compare SPARQL to an auto-generated SQL database as query backends, showing that OWL's structural features provide a substantial advantage over SQL DDL for LLM-driven query generation. A notable consideration for our demonstration domain is a requirement to run local LLMs on modest institutional hardware in support of privacy concerns for human subject data.

Keywords

Knowledge Graph, Natural Language Interface, Metadata Search, SPARQL, SQL, OWL, RDF, Large Language Model, Ontology Design, Neuroimaging, DICOM, BIDS, MRI

1 Introduction

Research facilities, enterprises, and government agencies accumulate large archives of domain-specific data with rich metadata, for example subject demographics, acquisition parameters, experimental configurations, and provenance records. Querying this metadata ad-hoc can require writing structured queries in SQL or SPARQL, which in turn requires both knowledge of the query language and the domain vocabulary. Many researchers lack the

expertise to effectively access the information they need without assistance.

Large language models (LLMs) can generate structured queries from natural language (NL), but their accuracy depends on how the underlying data schema is presented. On simple schemas like Spider [29], top models exceed 90% accuracy, but on real enterprise schemas with hundreds of tables, GPT-4o drops to 0% [4]. Sequeda et al. [20] find that reformulating the same enterprise questions over a knowledge graph raises accuracy from 16% to 54%. How domain knowledge is structured matters as much as which LLM model is used.

In this paper, we present an ontology-first process for enabling NL access to domain-specific metadata, a generic framework of tools to facilitate deployment, and measured results searching metadata on a large MRI image archive. The process begins with capturing the domain vocabulary and semantics in a formal OWL ontology [24] using readable entity names and semantic annotations. A domain-specific Extract-Transform-Load (ETL) pipeline maps archive metadata to the ontology, producing a knowledge graph (KG). Using the KG stored in RDF Turtle format, we load both a SPARQL database and, via automatic transformation, an SQL database, enabling a systematic comparison of both as targets for LLM-generated queries with equivalent data and NL questions. An LLM generates correct queries against either backend, zero-shot, with the full ontology or derived schema provided in full, in the system prompt.

The domain-specific ontology, competency and regression test cases, and a domain-specific prompt rider evolve together through iterative testing. During this process, the vocabulary and semantics of the domain are refined in increasing detail, progressively eliminating confusion and inaccuracies that plague even expert human communication.

We demonstrate this process on an MRI neuroimaging research archive [6] containing metadata for 80 active studies with 2,000 MRI experiments. Since this archive contains human subject data, privacy regulations (GDPR) require that all processing remain on institutional infrastructure rather than external LLM services. A practical objective is therefore to find the smallest model that achieves acceptable accuracy, keeping hardware costs manageable for institutions that may have modest GPU resources. We evaluate across 8 locally deployed LLM variants (8B to 35B parameters, including quantized and mixture-of-experts variants), showing that the best model achieves 100% accuracy on SPARQL and 57% on auto-generated SQL, with no fine-tuning, no retrieval augmentation, and no multi-agent orchestration [26, 30–32]. Stripping ontology annotations degrades SPARQL accuracy by 19 percentage points and SQL by 14, revealing the importance of semantic annotations for LLM-driven query generation.

Contributions:

- (1) The NLKGQ framework and ontology-first development process for enabling NL access to domain-specific meta-data, including ontology design principles, a KG builder pattern, and a reusable query server with web interface, demonstrated on neuroimaging research data
- (2) A systematic evaluation of LLM-driven SPARQL generation across 8 models, 8 ontology representations, and 768 configurations (over 16,000 runs), showing that ontology design is a dominant factor in accuracy
- (3) A generic OWL KG-to-SQL schema generator and data transformer enabling a systematic comparison of text-to-SPARQL vs text-to-SQL for the same NL questions
- (4) Analysis of the structural advantages OWL provides over SQL Data Definition Language (DDL) for LLM-based query generation

2 Related Work

2.1 Knowledge Graph Question Answering

Traditional Knowledge Graph Question Answering (KGQA) systems combine entity linking with relation prediction, mapping NL to structured queries through classification [9, 22]. Recent work applies LLMs to this problem through several strategies.

Fine-tuning approaches train models on SPARQL examples. SGPT [19] introduced generative pre-training for SPARQL. Zhang et al. [30] use GAIL (Generative Adversarial Imitation Learning) to fine-tune LLMs for low-resource KGQA, where the LLM acts as a generator producing SPARQL that a discriminator evaluates against expert demonstrations.

Multi-agent and pipeline approaches add orchestration layers. AGENTiGraph [32] deploys intent classification, task planning, and automatic knowledge integration across multiple agents, achieving 95% classification accuracy on a 3,500-query benchmark. CyberBOT [31] combines RAG with an ontology-based verification layer that constrains LLM outputs post-hoc.

In-context learning approaches provide examples at inference time. D’Abramo et al. [5] show that ICL with retrieved SPARQL examples can match fine-tuned models on KGQA benchmarks, though their approach requires example retrieval infrastructure. GRASP [26] uses the LLM to explore the knowledge graph at query time, searching for relevant IRIs and literals, achieving state-of-the-art results on Wikidata in a zero-shot setting.

All of these approaches build complexity to cope with ontologies they cannot change, typically public KGs like Wikidata [23] or DBpedia, where opaque identifiers provide few semantic hints to the LLM. Our approach differs in assuming the ontology is under our control. Domain-specific ontologies are typically smaller in vocabulary than public KGs, but may describe large volumes of data with a compact set of classes and properties.

2.2 Text-to-SQL and Schema Representation

Text-to-SQL has seen rapid progress on benchmarks like Spider [29], but performance depends on schema simplicity. BEAVER [4] shows that GPT-4o achieves 0% execution accuracy on real enterprise schemas with hundreds of tables, with failures traced to schema

retrieval and column mapping. Sequeda et al. [20] show that knowledge graphs provide a structural advantage, raising accuracy from 16% (SQL) to 54% (SPARQL) on the same enterprise questions. Rajkumar et al. [16] show that schema representation, how table and column names are presented, directly affects LLM accuracy for SQL generation, paralleling our findings for SPARQL. Wretblad et al. [28] show that adding column descriptions to SQL schemas improves text-to-SQL accuracy by over 20% on columns with uninformative names, validating the importance of annotations.

2.3 Text-to-SPARQL

LLM-based approaches to SPARQL generation began with sequence-to-sequence models [21]. Chain-of-thought prompting [1] and complex benchmarks like Spider4SPARQL [2] have advanced the field, but most work targets public KGs with opaque identifiers.

Closest to our work, Rasheed and Aguado [17] investigate prompt augmentation formats for domain-specific KGs, testing reduced and condensed ontology representations. However, they take the existing KG ontology as given and focus on fitting it into the prompt, rather than designing the ontology for LLM consumption from the start. Our work focuses on domain-specific ontologies of finite size that fit in the LLM context window. Our measurements show that departing from expressive naming of ontological elements hurts accuracy.

2.4 Our Focus

Prior work treats the ontology or database schema as a given and focuses on improving query generation. To our knowledge, no prior work addresses the end-to-end process: how to capture domain vocabulary in a formal ontology, build a knowledge graph from domain metadata via ETL, and enable LLM-driven NL querying, all as a reusable technique for domain practitioners.

Furthermore, while Sequeda et al. [20] compare LLM accuracy on SQL vs SPARQL across different schemas and datasets, no prior work compares SPARQL and SQL as query backends when both are derived from the same authoritative ontology, with the same readable naming, the same data, and the same LLM models. We provide this controlled comparison by automatically generating a relational schema from the domain ontology and evaluating both backends on the same competency questions.

3 Production Framework with Demonstration Domain

Our contribution has two main areas: a *development process* and a *reusable framework*. The development process is what a domain practitioner follows to enable NL access for a new domain: (1) capture domain vocabulary in an OWL ontology, (2) build a knowledge graph via ETL from domain data sources, and (3) iterate the ontology, competency questions, and prompt rider together until accuracy is acceptable. The reusable framework is the domain-agnostic infrastructure that supports this process: the NLKGQ query server, web application, and combinatorial test driver. The framework also supports production deployment. Both are described below, illustrated with our neuroimaging demonstration domain.

3.1 Capturing Domain Vocabulary

The first step is formalizing what domain experts already know: the entities, their properties, their relationships, and what they mean. We capture this in an OWL ontology [15, 24] following six design principles that cost nothing in formal expressiveness but measurably improve LLM accuracy:

- (1) **Full English words:** hasAcquisitionTime rather than acqT or AT
- (2) **Consistent naming pattern:** Properties follow has<Property> (e.g., hasSubjectId, hasRepetitionTime)
- (3) **Descriptive relationships:** isSubjectOfExperiment clearly indicates direction
- (4) **Explicit domain and range:** Property definitions specify which classes they connect
- (5) **Natural language annotations:** rdfs:comment, rdfs:label, and skos:altLabel provide context, readable names, and synonyms beyond the property URI
- (6) **No opaque codes:** Meaningful URIs throughout; no arbitrary or auto-generated identifiers

These are not cosmetic choices. They are engineering decisions made during ontology development. When building a new domain knowledge graph, readability costs little: the ontology must be designed regardless, and readable names are no harder to define than opaque ones. These principles also align with the FAIR (Findable, Accessible, Interoperable, Reusable) data guidelines [27], particularly Reusable: the ontology provides rich, domain-relevant metadata described with formal semantics, making both the data and the NL access layer reusable across tools and communities.

Consider a concrete example from our demonstration domain ontology (MRI Research Ontology, MRO):

```
mro:hasHandedness a owl:DatatypeProperty ;
  rdfs:label "Handedness"@en ;
  rdfs:comment "Subject handedness: Right, Left, Ambidextrous, or Unknown. (MRO)"@en ;
  rdfs:domain mro:Subject ;
  rdfs:range xsd:string .
```

The property name, label, comment, domain, and range together allow the LLM to use this property correctly without any prior exposure to our ontology. The comment tells the LLM what values to expect, enabling it to generate correct FILTER clauses. Contrast this with Wikidata’s wdt:P552 (handedness): the identifier P552 carries no semantic information, so the mapping must be learned through fine-tuning, few-shot examples, or graph exploration.

Frontier LLMs can assist domain experts in formalizing their vocabulary into OWL, lowering the barrier to ontology creation for practitioners without semantic web expertise.

Figure 1 shows the class and property structure of our demonstration domain ontology (MRO).

3.2 ETL and Knowledge Graph Construction

With the domain ontology defined, a domain-specific ETL pipeline collects metadata and populates a KG. In our demonstration domain, metadata is extracted from DICOM headers, NIfTI sidecar files [8, 11], Siemens TWIX raw data headers, an experiment registry, and a participant management system [13]. Our Python pipeline uses

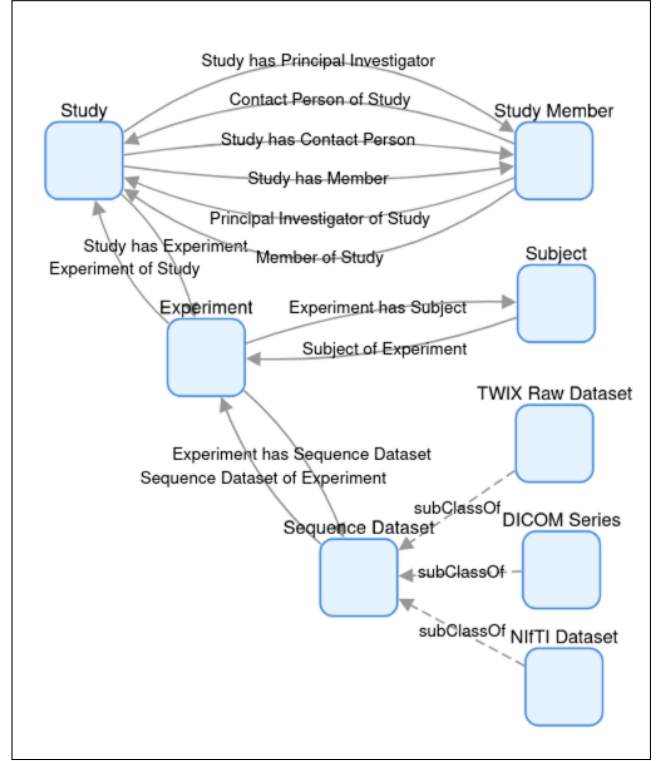


Figure 1: MRI Research Ontology (MRO) class diagram. Classes are connected by object properties (arrows).

rdflib [18] to map extracted fields to MRO classes and properties, producing RDF triples. The resulting KG contains approximately 10 million triples derived from a 180TB MRI archive.

Each new domain requires its own ETL logic, but the pattern is the same: identify metadata sources, write extraction code, map fields to ontology terms, generate RDF. The ETL pipeline can also anonymize sensitive fields and compute aggregate properties (e.g., experiment duration from individual acquisition timestamps) during the transformation step. Rather than relying on an OWL reasoner at query time, the ETL pipeline materializes all implied relationships as concrete RDF triples (e.g., generating both directions of inverse properties and explicit class memberships) during KG construction; this is sufficient for the OWL constructs we use. The ontology guides what to extract and how to name it.

The resulting KG is loaded into a SPARQL server, in our case Apache Jena Fuseki [3], where it can be queried using SPARQL 1.1 [25] via the web interface or HTTP API. This keeps the SPARQL server lightweight and query latency predictable.

3.3 Iterative Development

Ontology development is not a one-time design step. The ontology, competency questions, and domain-specific prompt rider co-evolve through testing. Running test cases against the system reveals gaps that feed back into all three:

- A property name that confuses the LLM gets renamed in the ontology (e.g., adding skos:altLabel “pseudonym” to

hasSubjectId when users say “pseudonym” instead of “subject ID”)

- A recurring LLM mistake gets addressed by a new directive in the domain-specific prompt rider (e.g., “return experiment IDs, not instance URIs”)
- A new user question that the system handles poorly becomes a new competency test case

This cycle is lightweight: each iteration is a small change to one of the three artifacts, followed by re-running the test suite. It requires domain knowledge but no specialized technical skills. The result is a progressively refined system where the ontology captures the domain vocabulary, the test cases capture the query patterns, and the rider captures the conventions that naming alone cannot express.

3.4 NLKGQ Query Server and Web Application

The NLKGQ server receives an NL question and constructs a system prompt containing: (1) generic SPARQL generation instructions (output format, error handling, query patterns to prefer or avoid), (2) the complete domain ontology in OWL Turtle format, and (3) a small domain-specific rider with conventions particular to the knowledge graph (e.g., preferred identifier patterns, directives addressing recurring LLM mistakes). The user’s NL question is appended as the user message, and the LLM generates SPARQL.

Logic in the harness handles practical issues of the LLM response: stripping chain-of-thought <think> blocks, extracting SPARQL from markdown, fixing PREFIX declarations using the ontology as ground truth, and optionally retrying failed queries by sending the error back to the LLM. Retrying failed queries is technically not zero-shot; however, it primarily helps smaller models recover from syntax errors, while the best-performing models rarely trigger it.

The web application provides an interface where end users type NL questions and receive query results, with the generated SPARQL shown for transparency. Features include prefix compression that strips verbose RDF URIs from query results, displaying data with readable prefixed names instead (with a toggle for full URIs when needed), CSV download of results, an LLM-powered “Explain” button that describes what the generated SPARQL does in plain language, Python script export for rerunning queries outside the web interface, an interactive ontology graph visualization, and a “Save Test Case” function that packages a query with its results for use in the evaluation framework. Displaying the generated query builds user trust and helps researchers learn which phrasings produce useful results. The system is deployed at a neuroimaging research facility where neuroscience researchers with varying levels of query expertise can search their metadata directly. Figure 2 shows the web interface processing a natural language query.

4 Research Methods and Experimental Setup

To rigorously evaluate our design decisions, we developed additional research infrastructure beyond the production system. Some of this infrastructure also supports the iterative development process (Section 3.3), serving both production and research needs.

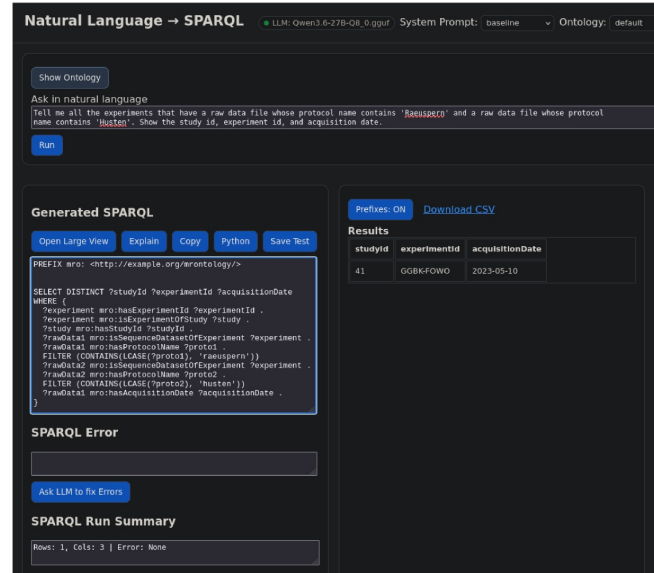


Figure 2: The NLKGQ web interface. The user types a natural language question (top), the system generates and displays the SPARQL query (left), and shows the result table with prefix-compressed URIs (right).

4.1 Test Driver Framework

The test driver is the foundation of both domain development and research evaluation. Given a competency question (NL), a reference SPARQL query, and expected results, the driver calls the LLM, extracts the generated query, executes it against the appropriate backend, and compares results. The driver supports combinatorial sweeps across the following parameters for each competency question:

- **LLM model:** different model families, sizes, and quantization variants
- **Temperature:** controls generation randomness (e.g., 0.0, 0.2)
- **System prompt:** different prompt formulations and domain-specific riders
- **Ontology representation:** full Turtle, no-comments, compact, abstract, etc.
- **Query backend:** SPARQL or auto-generated SQL (with or without column comments)
- **Repetitions:** multiple runs per configuration to assess variability
- **Fix-retry policy:** number of LLM correction attempts on syntax errors

Results are logged to CSV with per-query timing, match status, and fix-retry outcomes.

A walker script orchestrates the driver across all test cases and configurations, with parallel execution across models. For the SQL evaluation, a separate test driver executes against PostgreSQL with tolerant comparison (column name normalization, order-independent row matching, and URI prefix stripping) to

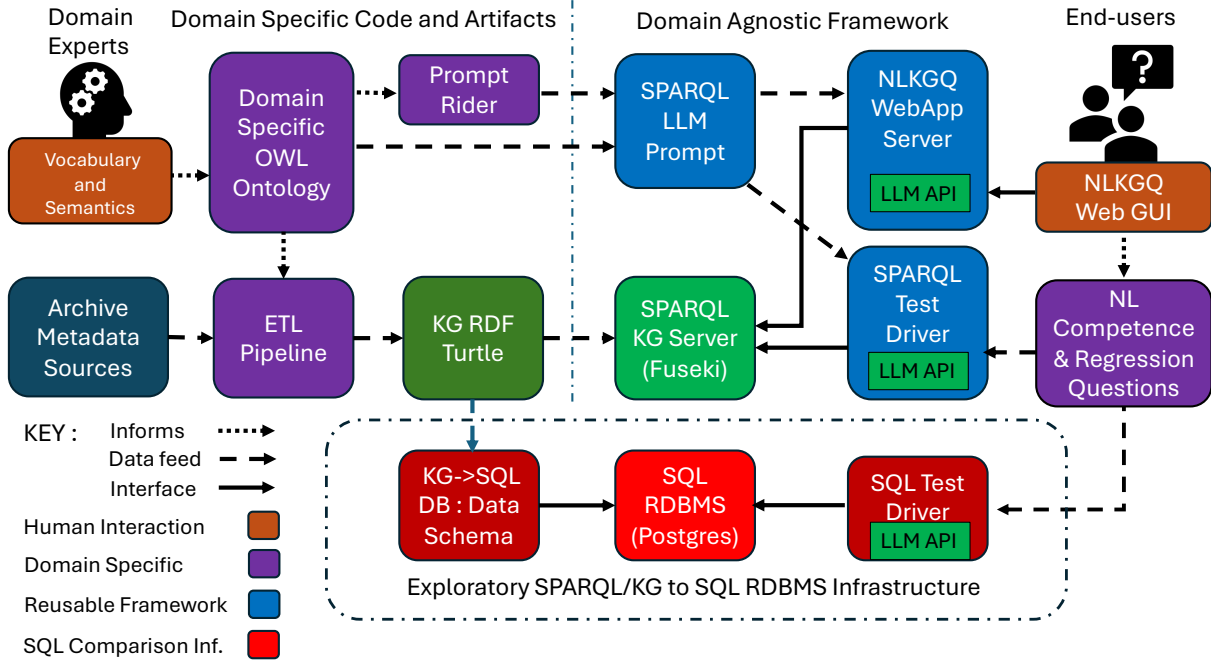


Figure 3: Framework architecture showing production components (solid) and research infrastructure (dashed).

handle differences between SQL output and SPARQL reference data.

The test driver is used both for iterative domain development (Section 3.3) and for the research evaluations described below.

4.2 Virtual Ontology Representations

To measure which ontology features different LLMs rely on, we evaluate eight representations of the same ontology, generated automatically from the full OWL Turtle:

- **default:** Full OWL Turtle with domain, range, comments, labels
- **no-comments:** Turtle with `rdfs:comment` and `rdfs:label` stripped
- **compact:** One-line-per-property listing class memberships
- **compact-typed:** Compact with datatype annotations retained
- **compact-grouped:** Properties grouped by domain class
- **raw-graph:** RDF triples in N-Triples format
- **abstract-dict:** Dictionary mapping generic keys to values
- **abstract-graph:** Graph with anonymized node/edge labels

These form an ablation study: moving from *default* to *abstract-graph* progressively strips semantic information, revealing which features the LLM relies on.

4.3 Automatic Generation of SQL Database

We developed a generic OWL-to-SQL converter that reads the domain ontology and automatically derives a PostgreSQL 17 schema: each OWL class becomes a table, each datatype property becomes a column (typed from `rdfs:range`), and object properties become foreign keys (direction inferred from `owl:inverseOf`). Ontology annotations (`rdfs:comment`, `rdfs:label`, `skos:altLabel`) are converted to SQL column comments. The same KG data is loaded into the relational schema (6 tables, 182 columns for the largest table).

A system prompt analogous to the SPARQL prompt provides the Data Definition Language (DDL) schema (with or without comments) and instructs the LLM to generate SQL instead of SPARQL. A SQL test driver evaluates the same competency questions against PostgreSQL using tolerant comparison to match SQL results against SPARQL reference data.

4.4 Models and Hardware

We evaluate eight model variants from the Qwen3 family, spanning 8B to 35B parameters, including dense and mixture-of-experts (MoE) architectures, and quantized variants for production deployment:

Models are served on two GPU systems: AMD Instinct MI300A APUs (128GB HBM3 per APU) at a shared HPC facility [14] via vLLM [10], and an institutional 4× Quadro RTX 5000 system (64GB total VRAM) via llama.cpp for the Q8 variant. The Quadro RTX 5000 based system (AMD CPU, 256GB main memory, 4×16GB GPU

Table 1: Models evaluated. All served locally. HW: A=AMD MI300A APU (HPC), N=4×NVIDIA RTX5000 (institutional).

Model	Params	Type	Ctx	Server	HW
Qwen3-8B	8B	Dense	41K	vLLM	A
Qwen3-14B	14B	Dense	41K	vLLM	A
Qwen3-Coder-30B-A3B	30B/3B	MoE	64K	vLLM	A
Qwen3.6-27B	27B	Dense	64K	vLLM	A
Qwen3.6-27B-FP8	27B	Dense/FP8	64K	vLLM	A
Qwen3.6-27B-Q8_0	27B	Dense/Q8	32K	llama.cpp	N
Qwen3.6-35B-A3B	35B/3B	MoE	64K	vLLM	A
Qwen3.6-35B-A3B-FP8	35B/3B	MoE/FP8	64K	vLLM	A

VRAM) is vintage 2020 hardware and serves as our production deployment target. Context windows range from 32K to 64K tokens (Table 1). The full OWL Turtle ontology (80KB, approximately 18K tokens) and the SQL schema with comments are each provided in the system prompt, requiring models that can handle substantial context alongside the generation task. Even the smallest context window (32K for the Q8 model) provides ample headroom, and no context overflow errors were observed in any run. The same models are used for both SPARQL and SQL evaluation.

4.5 Evaluation Protocol

Over twenty competency and regression questions were developed with domain experts, covering filtered queries (“Subjects over age 42 in study 42”), multi-entity joins (“Experiments with subject from study EPEQ-7ITX”), and aggregations with sorting (“All datasets for a study, sorted by acquisition time”). Each test case comprises an NL question, a reference SPARQL query, and expected results in SPARQL Results JSON format. The web application’s “Save Test Case” function generates these artifacts directly from a successful query, enabling domain experts to create new test cases without manual JSON editing. The primary metric is *result match*: exact equality of the query result set, with tolerant comparison to handle variable naming differences.

SPARQL evaluation. Each test case is evaluated across all combinations of model, ontology representation, system prompt variant, and temperature setting: 8 models × 8 ontology representations × 3 prompts × 4 temperatures, yielding over 16,000 runs.

SQL evaluation. The same test cases are evaluated across 8 models × 2 schema variants (with and without column comments) × 4 temperatures.

For both backends, when a generated query produces a syntax error, the harness sends the query and error message back to the LLM for up to two correction attempts. As discussed in Section 3.4, this is technically not zero-shot, but it is rarely used with the LLMs chosen for production deployment. LLM generation time is recorded for each run.

4.6 Example Queries and Generated SPARQL

To illustrate what the system produces, we show three competency questions of increasing complexity with the SPARQL generated by Qwen3.6-27B (default ontology, zero-shot).

Simple lookup. NL: “Show all the experiments by ID in study 101.”

```
SELECT DISTINCT ?ExperimentId WHERE {
  ?study mro:hasStudyId "101" .
  ?experiment mro:isExperimentOfStudy ?study .
  ?experiment mro:hasExperimentId ?ExperimentId .
}
```

Multi-entity join. NL: “List all experiment IDs for study 101. Show the subject ID, handedness, age, and acquisition date for each.”

```
SELECT DISTINCT ?ExperimentId ?SubjectId
           ?Handedness ?PatientAge ?AcquisitionDate
WHERE {
  ?exp mro:isExperimentOfStudy ?study .
  ?study mro:hasStudyId "101" .
  ?exp mro:hasExperimentId ?ExperimentId .
  ?exp mro:aExperimentHasSubject ?subj .
  ?subj mro:hasSubjectId ?SubjectId .
  ?subj mro:hasHandedness ?Handedness .
  ?exp mro:hasPatientAge ?PatientAge .
  ?exp mro:hasExperimentBeginDate ?AcquisitionDate .
}
```

Filtered aggregation. NL: “For study 42, find experiments where the subject is older than 42. List only experiments before June 2025.”

```
SELECT DISTINCT ?ExperimentId ?PatientAge WHERE {
  ?exp mro:isExperimentOfStudy ?study .
  ?study mro:hasStudyId "42" .
  ?exp mro:hasExperimentId ?ExperimentId .
  ?exp mro:hasPatientAge ?PatientAge .
  ?exp mro:hasExperimentBeginDate ?date .
  FILTER(xsd:integer(?PatientAge) > 42)
  FILTER(?date < "2025-06-01"^^xsd:date)
}
```

All three queries are generated zero-shot from the NL question and the ontology alone. The LLM correctly uses ontology property names, traverses relationships via inverse properties, and applies appropriate type casting in filters. PREFIX declarations are omitted above for brevity.

Table 2 lists all competency questions used in the evaluation.

5 Results

5.1 SPARQL: Model Comparison

Table 3 shows the best SPARQL configuration for each model, reporting the full configuration (ontology representation, temperature, system prompt) that achieved the highest accuracy. Short model identifiers (ID column) are used in subsequent tables.

Both 27B dense variants (full-precision and Q8 quantized) achieve 100% accuracy on the full Turtle ontology. The Q8 model on older institutional hardware (N) is 1.8× slower but equally accurate as the full-precision model on HPC hardware (A). The 35B MoE models peak at 57–62%, well below the 27B dense models despite having more parameters, suggesting that MoE architectures may be less effective for structured generation tasks requiring precise schema adherence. Model scale matters within architectures: accuracy improves from 29% (8B) through 38% (14B) to 100% (27B dense).

Table 2: Competency questions by category. L=lookup, J=join, F=filter, A=aggregation, S=sort, O=ontology.

#	Description	Type
1	List experiments in a study	L
2	Find experiments for a subject by pseudonym	L
3	Find all experiments for the subject in a given experiment	J
4	Same as 3, rephrased as two-part question	J
5	Experiments matching two protocol name patterns	J,F
6	Experiment details with subject demographics and date	J
7	Dataset names with acquisition date and time	L,S
8	All properties and values on a specific dataset instance	O
9	Acquisition dates for all experiments in a study, sorted	L,S
10	Experiments filtered by subject age and date range	F
11	Subject demographics (language, degree, handedness, etc.)	J
12	Sequence datasets matching coil name substring	F
13	Same as 12, exact coil name match	F
14	Experiments after a time of day, sorted by duration	F,S
15	Experiments in a date and duration range with study info	F,J,S
16	Experiments using a specific coil with study and date	J,F
17	Studies with scanner types and total experiment hours	A,J,S
18	Ontology introspection: classes, properties, and types	O
19	Coils used in a date range with experiment counts	A,F
20	Top N coils by usage count in a year	A,F,S
21	Datasets on a specific experiment, sorted by time	L,S

5.2 SPARQL: Ontology Representation Impact

Table 4 shows the best configuration for each ontology representation, reporting the full configuration that achieved the highest accuracy.

Only *default* (the complete OWL Turtle ontology with all annotations, no ablation) reaches 100%. Stripping `rdfs:comment` and `rdfs:label` annotations drops accuracy to 81%. Compact representations that retain property names achieve 62–71%. Abstract representations that replace readable entity names with generic identifiers collapse to 10–14%, confirming naming as a dominant factor in accuracy. The Tokens column shows the prompt size for each representation, ranging from 1.7K (compact) to 17.6K (default).

Property names are essential: the jump from *compact* (62%) to *abstract-dict* (19%) shows the LLM uses name semantics, not just graph structure. **Annotations** matter: stripping comments costs 19 percentage points for SPARQL, and as we show below, comments are even more critical for SQL. **Domain and range** declarations help: *compact-typed* (71%) outperforms *compact* (62%), suggesting type information reduces cross-class property confusion.

At our ontology’s scale (80KB), the full Turtle fits comfortably within 32K-token context windows, so there is no practical reason to sacrifice accuracy for compactness. Looking forward, many smaller models now support substantially larger context windows (64K

to 256K tokens), suggesting these methods will scale to domains requiring more complex ontologies.

5.3 SPARQL: System Prompt and Temperature

Tables 5 and 6 show the best configurations for each system prompt variant and temperature setting.

Temperature and system prompt have modest effects compared to model choice and ontology representation. All three prompts reach 100% with the 27B dense model at $t=0.0$, and all four temperatures reach at least 95% when paired with the right model and the *default* ontology. The baseline prompt (the simplest of the three) achieves 100%, suggesting that with a well-designed ontology, elaborate prompting is unnecessary.

5.4 Auto-SQL Results

Table 7 shows auto-SQL accuracy with and without ontology-derived column comments.

The best model achieves 57% on auto-SQL compared to 100% on SPARQL. Ontology-derived column comments add 3 cases for the best model (43% without, 57% with), a considerably larger effect than stripping annotations from the SPARQL ontology (100% to 71%). The Q8 model (Q36.27B.Q) on institutional hardware achieves the highest SQL accuracy, suggesting that the chain-of-thought reasoning enabled in llama.cpp benefits the more complex SQL generation task.

5.5 Practical Deployment on Modest Hardware

For SPARQL with the *default* ontology, both 27B dense variants (Q36.27B.D and Q36.27B.Q) achieve 100% accuracy across multiple prompt and temperature configurations, while the FP8 variant (Q36.27B.F) reaches 95%.

The Q8-quantized model (Q36.27B.Q) running on 4× Quadro RTX 5000 GPUs (a machine several generations old) achieves 100% SPARQL accuracy and the highest SQL accuracy (57%), demonstrating that the approach does not require recent-generation GPUs. This matters for institutions with privacy constraints (e.g., GDPR for human subject data) that must deploy LLMs locally [27]. Published benchmarks [7, 12] indicate that current Apple Silicon hardware with 128GB unified memory can run quantized 27B models at 25–55 tokens per second via MLX or llama.cpp, with systems starting under US\$3,000. The low idle power consumption of these systems makes them practical for always-on deployment, and the large unified memory leaves headroom for growing ontologies requiring larger context windows.

6 Discussion

6.1 The Ontology as Single Source of Truth

The central finding is that co-designing an LLM-friendly domain ontology along with the ETL logic building the KG and the domain-specific system prompt rider yields far better results than designing any one of these components independently, or worse, accepting existing components as a given.

A related finding is that ontology-driven SPARQL queries are easier for an LLM to generate accurately than an equivalent SQL query on a relational model.

Table 3: Best SPARQL configuration per model (21 competency questions). The ID column provides short identifiers used in subsequent tables: Q30/Q36=Qwen generation, xxB=parameters, D=dense, M=MoE, F=FP8, Q=Q8, C=Coder. HW from Table 1.

Model ID	Model Name	Acc%	Ontology	Temp	Prompt	Avg Time	Tokens	HW
Q36.27B.D	Qwen3.6-27B	100	default	0.0	baseline	8.1s	17.6K	A
Q36.27B.Q	Qwen3.6-27B-Q8_0	100	default	0.0	procedural	14.6s	17.7K	N
Q36.27B.F	Qwen3.6-27B-FP8	95	default	0.0	guardrails	7.9s	17.8K	A
Q30.30B.C	Qwen3-Coder-30B-A3B	62	compact-grouped	0.2	procedural	1.0s	1.9K	A
Q36.35B.F	Qwen3.6-35B-A3B-FP8	62	default	0.6	guardrails	25.8s	17.8K	A
Q36.35B.M	Qwen3.6-35B-A3B	57	default	0.2	procedural	24.2s	17.7K	A
Q30.14B.D	Qwen3-14B	38	default	0.0	procedural	1.8s	16.8K	A
Q30.08B.D	Qwen3-8B	29	default	0.6	procedural	0.9s	16.8K	A

Table 4: Best SPARQL configuration per ontology representation (21 questions). Model IDs from Table 3.

Ontology	Acc%	Temp	Prompt	Avg Time	Tokens	Model ID
default	100	0.0	baseline	8.1s	17.6K	Q36.27B.D
no-comments	81	0.0	procedural	5.5s	7.6K	Q36.27B.D
compact-typed	71	0.0	procedural	3.0s	1.9K	Q36.27B.D
compact-grouped	71	0.2	procedural	2.9s	1.9K	Q36.27B.D
compact	62	0.0	procedural	3.3s	1.7K	Q36.27B.D
raw-graph	52	0.4	guardrails	3.5s	4.1K	Q36.27B.D
abstract-dict	19	0.0	guardrails	20.9s	5.9K	Q36.35B.M
abstract-graph	10	0.0	guardrails	31.8s	3.8K	Q36.35B.F

Table 5: Best SPARQL configurations per system prompt (21 questions). All use *default* ontology.

Prompt	Acc%	Model ID	Temp	Avg Time
baseline	100	Q36.27B.D	0.0	8.1s
baseline	100	Q36.27B.Q	0.2	14.2s
guardrails	100	Q36.27B.D	0.0	8.3s
procedural	100	Q36.27B.D	0.0	8.5s
procedural	100	Q36.27B.Q	0.0	14.6s

Table 6: Best SPARQL configurations per temperature (21 questions). All use *default* ontology.

Temp	Acc%	Model ID	Prompt	Avg Time
0.0	100	Q36.27B.D	baseline	8.1s
0.0	100	Q36.27B.D	guardrails	8.3s
0.0	100	Q36.27B.D	procedural	8.5s
0.0	100	Q36.27B.Q	procedural	14.6s
0.2	100	Q36.27B.Q	baseline	14.2s
0.4	95	Q36.27B.F	guardrails	7.8s
0.6	95	Q36.27B.F	procedural	8.0s

Experience with a neuroimaging demonstration domain shows that end users can create well-formed natural language queries where they would not take the time to learn and write SPARQL or SQL queries.

Table 7: Best auto-SQL accuracy per model (21 questions, wide-table schema). Model IDs from Table 3.

Model ID	With comments	No comments
Q36.27B.Q	12/21 (57%)	9/21 (43%)
Q36.27B.D	11/21 (52%)	7/21 (33%)
Q36.27B.F	11/21 (52%)	8/21 (38%)
Q36.35B.M	11/21 (52%)	6/21 (29%)
Q36.35B.F	11/21 (52%)	7/21 (33%)
Q30.14B.D	9/21 (43%)	7/21 (33%)
Q30.08B.D	8/21 (38%)	6/21 (29%)
Q30.30B.C	8/21 (38%)	7/21 (33%)

Finally, we have found that we can run this service on local LLMs using older or less expensive hardware. It is likely that for domains without privacy constraints, cloud-based frontier LLMs with larger context windows would be faster and more capable.

The complex pipelines in prior work [26, 30–32] exist for a reason: they cope with ontologies not designed for LLM consumption. When the ontology *can* be designed, which is the case for every new domain metadata project, the simpler path is available.

6.2 SPARQL vs SQL: What the Ontology Provides

The gap between SPARQL and SQL accuracy (100% vs 57%) shows that while readable naming helps both, OWL provides structural advantages that SQL DDL lacks:

Annotations. Stripping `rdfs:comment` from the SPARQL ontology drops accuracy from 100% to 81%. Stripping the equivalent

Table 8: Per-query comparison using each backend’s best configuration. SPARQL: Q36.27B.D, default ontology, $t=0.0$, baseline. Auto-SQL: Q36.27B.Q, with comments, $t=0.0$. Q# from Table 2. Time is LLM generation in seconds.

Q#	Query	SPARQL		SQL	
		OK	s	OK	s
1	Experiments in a study	✓	4	✓	4
2	Experiments for a subject	✓	5	✓	4
3	All experiments for a subject	✓	5	✓	5
4	Same as Q3, rephrased	✓	6	✓	8
5	Two protocol name patterns	✓	9	✓	8
6	Experiment details with demographics	✓	10	✓	9
7	Dataset name, acq date and time	✓	7	×	6
8	All properties on a dataset	✓	5	×	113
9	Acquisition dates sorted	✓	7	×	7
10	Age filter with date range	✓	9	✓	8
11	Subject demographics for a study	✓	10	×	8
12	Datasets matching coil substring	✓	9	✓	12
13	Datasets matching exact coil	✓	8	✓	9
14	Experiments after 4pm, sorted	✓	9	×	10
15	Experiments in duration range	✓	12	✓	13
16	Experiments for a coil with date	✓	9	×	8
17	Studies with scanner hours	✓	11	×	12
18	Ontology classes and properties	✓	15	×	8
19	Coils used in a date range	✓	10	✓	12
20	Top N coils by experiment count	✓	9	×	10
21	Datasets on experiment, sorted	✓	6	✓	7
Total		21/21		12/21	

SQL column comments drops accuracy from 57% to 43%. Annotations are critical for both backends, consistent with Wretblad et al. [28].

Domain and range declarations. In our conversion, `rdfs:range` determines SQL column types and `rdfs:domain` determines table placement. These are preserved in the DDL, but the explicit class-level grouping visible in the Turtle ontology is flattened into a column list that the LLM must parse without the same semantic scaffolding.

Inverse properties. In SPARQL, `owl:inverseOf` allows the LLM to traverse any relationship in either direction. A query can use `mro:aStudyHasExperiment` or `mro:isExperimentOfStudy` interchangeably. In SQL, there is one foreign key, and the LLM must know which table owns it and JOIN from the correct side. The SQL system prompt must explicitly encode join directions, while the SPARQL ontology encodes them structurally.

Entity-Attribute-Value schema. As an alternative to the wide-table SQL schema (one column per property, NULLs where properties are absent), we also evaluated an Entity-Attribute-Value (EAV) representation: a narrow three-column table (`entity_id`, `attribute`, `value`) per class, where each row stores a single fact. This schema was automatically derived from the same ontology. EAV accuracy reached only 11/21 (52%) with the best model and comments, dropping to 2/21 (10%) without comments. A key difficulty is that relationship attributes in EAV store prefixed entity identifiers (e.g., `study_101`) rather than clean values (`101`) that the LLM can reason about directly. In SPARQL, the LLM never sees internal identifiers because it operates on variables and property patterns. In wide-table SQL, foreign keys reference clean primary keys. EAV exposes internal naming conventions to the query generator, requiring a domain-specific prompt rider to teach the convention. This represents a structural disadvantage that readable naming alone cannot overcome.

6.3 Lessons Learned

Three findings were counterintuitive:

MoE architectures underperformed dense models. The 35B MoE variants peaked at 52%, well below the 27B dense models at 100%, despite having more total parameters. For structured generation tasks requiring precise schema adherence, dense models appear to be the better choice at a given compute budget.

Fine-tuning hurt. Fine-tuning a smaller model (8B) on domain-specific examples achieved only 22%, compared to 43% for the same base model zero-shot. While our fine-tuning exploration was limited, this suggests that the ontology-in-context approach may already provide the domain knowledge that fine-tuning attempts to bake in.

The simplest prompt won. The baseline prompt, with the fewest instructions, achieved the highest peak accuracy. More elaborate prompts (guardrails, procedural) performed comparably but not better, suggesting that with a well-designed ontology, the LLM needs minimal additional guidance.

Additional findings: **Full ontology in context** consistently produced the best results across all models and configurations. **Readable naming** had the largest measurable effect on accuracy. **Quantization** (Q8 GGUF and FP8) preserved accuracy completely. **Automatic OWL-to-SQL conversion** derived a working relational schema directly from the ontology with no manual tuning.

The test driver is essential infrastructure. The combinatorial test driver (Section 4.1) is not only research tooling; it is the engine that drives ontology evolution. The ontology, prompt rider, and competency questions co-evolve: each iteration of any of these three artifacts is validated by re-running the full suite, making regressions immediately visible. Without this tight feedback loop, the co-design process described in Section 3.3 would be more difficult. In our experience, ontology evolution slows over time but does not halt as the vocabulary and semantics are better resolved.

6.4 Limitations

Our evaluation covers a single metadata domain. While the ontology design principles are domain-agnostic, generalization to other domains needs validation. The test set of 20+ competency questions, while developed with domain experts, is small; expansion is ongoing. Because the competency questions co-evolved with the ontology and prompt rider, accuracy on novel end-user queries may differ from test set performance. The complexity of the ontology is bounded by the available LLM context window, with some promise shown for token-dense encodings. The volume of metadata under management is bounded by the capabilities of the SPARQL server. We use Jena Fuseki in a Docker container, but for larger archives a heavy-duty commercial solution could be used without changing the architecture. All models are from the Qwen3 family and results may differ with other model families. We have not compared against cloud-hosted models (GPT-4, Claude) due to institutional data privacy constraints.

6.5 Future Work

- **Cross-domain validation:** Applying the development process and framework to additional domains. Here we are open to discussing collaboration.

- **Expanded MRI archive use:** The MRO ontology builds on DICOM and BIDS standards shared across MRI research sites. With site-specific ETL adaptation, the same ontology and framework could serve other neuroimaging archives.
- **Expanded test cases:** Growing the competency question set, including more complex query patterns.
- **Multi-turn refinement:** Supporting follow-up questions where the LLM refines a previous query based on user feedback, making the web application more conversational.
- **LLM Model diversity:** Evaluating non-Qwen model families and cloud-hosted models where domain privacy constraints permit.

7 Conclusion

We have presented a reusable framework and development process for natural language access to domain-specific metadata. A key insight is that capturing domain vocabulary and semantics in a well-designed OWL ontology enables LLM-driven query generation against both SPARQL and SQL backends, with no fine-tuning, no retrieval augmentation, and no multi-agent orchestration.

The ontology serves as the single source of truth: it defines the domain vocabulary, informs the ETL pipeline, and provides the LLM with schema context for SPARQL queries. To explore SPARQL vs SQL for this work, we automatically generate a relational schema, transform and load the KG into a PostgreSQL database, and evaluate the competency questions via SQL, enabling a direct comparison. On our MRI neuroimaging metadata benchmark, NL text-to-SPARQL achieves 100% accuracy on our competency/regression question set while the analogous NL text-to-SQL achieves 57%. An ablation study across eight ontology representations confirms that ontology design, particularly readable naming and semantic annotations, is the dominant factor in accuracy.

The framework and development process are intended to be reusable. The framework is designed to avoid domain-specific components. The iterative development process outlined here (ontology, competency questions, and prompt rider evolving together) requires domain expertise but no machine learning, database, or programming skills. We have demonstrated the process and deployed the framework at a major neuroscience institute enabling end-user natural language search of MRI metadata on a growing image archive.

We intend to open source this work including the MRI metadata domain and the framework, and we welcome collaboration on application to new domains.

GenAI Usage Disclosure

Generative AI tools were used to assist with manuscript editing and $\LaTeX$ formatting. Generative AI was also used to assist with coding, particularly the web server where we have little expertise available. The system described in this paper uses locally deployed LLMs (Qwen3 family) for SPARQL and SQL query generation, which is the subject of this research. All scientific content, experimental design, analysis, and conclusions are the work of the authors.

References

- [1] Debayan Banerjee, Sushil Awale, Ricardo Usbeck, and Chris Biemann. 2024. Generating SPARQL from Natural Language Using Chain-of-Thoughts

- Prompting. In *Proceedings of SEMANTICS*.
- [2] Catherine Brei, Annika Kraft, Marco Danner, Florian Vogel, and Jens Lehmann. 2023. Spider4SPARQL: A Complex Benchmark for Evaluating Knowledge Graph Question Answering Systems. In *Proceedings of ISWC*.
- [3] Jeremy J. Carroll, Ian Dickinson, Chris Dollin, Dave Reynolds, Andy Seaborne, and Kevin Wilkinson. 2004. Jena: Implementing the Semantic Web Recommendations. In *Proceedings of the 13th International World Wide Web Conference (WWW)*. ACM, 74–83. doi:10.1145/1013367.1013381
- [4] Peter Baile Chen, Fabian Wenz, Yi Zhang, Devin Yang, Justin Choi, Nesime Tatbul, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2024. BEAVER: An Enterprise Benchmark for Text-to-SQL. *arXiv preprint arXiv:2409.02038* (2024).
- [5] Jacopo D’Abramo, Andrea Zugarini, and Paolo Torroni. 2025. Investigating Large Language Models for Text-to-SPARQL Generation. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for NLP (KnowledgeNLP)*. Association for Computational Linguistics, 66–80.
- [6] Blake G. Fitch. 2022. MrData: An iRODS Based Human Research Data Management System. In *Proceedings of the iRODS User Group Meeting*. Leuven, Belgium.
- [7] Jeff Geerling. 2026. AI/LLM Benchmarks (llama.cpp and Ollama). <https://github.com/geerlingguy/ai-benchmarks>.
- [8] Krzysztof J Gorgolewski et al. 2016. The brain imaging data structure, a format for organizing and describing outputs of neuroimaging experiments. *Scientific Data* 3 (2016), 160044. doi:10.1038/sdata.2016.44
- [9] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. 2021. Knowledge Graphs. *Comput. Surveys* 54, 4 (2021), 1–37.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*.
- [11] Xiangrui Li, Paul S Morgan, John Ashburner, Jolinda Smith, and Christopher Rorden. 2016. The first step for neuroimaging data analysis: DICOM to NIfTI conversion. *Journal of Neuroscience Methods* 264 (2016), 47–56. doi:10.1016/j.jneumeth.2016.03.001
- [12] LLM Check. 2026. Apple Silicon LLM Benchmarks: Real tok/s by Model, Chip and Quantization. <https://llmcheck.net/benchmarks>.
- [13] Karolina Mader and Maike Kleemeyer. 2023. Castellum: A Data Protection-Compliant Web Application for the Subject Management of Human Science Studies. In *Proceedings of the Conference on Research Data Infrastructure (CoRDI)*, Vol. 1. doi:10.52825/cordi.v1i.325
- [14] Max Planck Computing and Data Facility. 2025. Viper-GPU User Guide. <https://docs.mpcdf.mpg.de/doc/computing/viper-gpu-user-guide.html>. 228 nodes, 2× AMD Instinct MI300A APUs per node, 128 GB HBM3 per APU.
- [15] Natalya F. Noy and Deborah L. McGuinness. 2001. *Ontology Development 101: A Guide to Creating Your First Ontology*. Technical Report. Stanford University. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- [16] Nitarshan Rajkumar, Raymond Li, and Dzmitry Baber. 2022. Evaluating the Text-to-SQL Capabilities of Large Language Models. *arXiv preprint arXiv:2204.00498* (2022).
- [17] Mohammed H. Rasheed and Marina Aguado. 2025. LLM-Based Natural Language to SPARQL Translation over Domain-Specific Knowledge Graph. *Knowledge Organization* 52, 8 (2025). doi:10.31083/KO42705
- [18] RDFLib Team. 2024. rdflib: A Python library for working with RDF. <https://github.com/RDFLib/rdflib>.
- [19] Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teuber, Liubov Kovriguina, and Jens Lehmann. 2022. SGPT: A Generative Approach for SPARQL Query Generation From Natural Language Questions. *IEEE Access* 10 (2022), 70712–70723. doi:10.1109/ACCESS.2022.3188714
- [20] Juan Sequeda, Dean Briggs, Nithish Mihindukulasooriya, and Oscar Corcho. 2023. A Benchmark to Understand the Role of Knowledge Graphs on Large Language Model’s Accuracy for Question Answering on Enterprise SQL Databases. *arXiv preprint arXiv:2311.07509* (2023).
- [21] Tommaso Soru, Edgard Marx, Diego Moussallem, Gustavo Publico, André Valdes-tilhas, Diego Esteves, and Ciro Baron Neto. 2017. SPARQL as a Foreign Language. In *SEMANTICS*.
- [22] Ricardo Usbeck, Ria Hari Gusmita, Axel-Cyrille Ngonga Ngomo, and Muhammad Saleem. 2018. 9th Challenge on Question Answering over Linked Data (QALD-9). In *Semantic Web Challenges*.
- [23] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85. doi:10.1145/2629489
- [24] W3C. 2012. OWL 2 Web Ontology Language Primer. <https://www.w3.org/TR/owl2-primer/>.
- [25] W3C. 2013. SPARQL 1.1 Query Language. <https://www.w3.org/TR/sparql11-query/>.
- [26] Sebastian Walter and Hannah Bast. 2026. GRASP: Generic Reasoning And SPARQL Generation across Knowledge Graphs. In *Proceedings of ECIR*.

- [27] Mark D Wilkinson et al. 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3 (2016), 160018.
- [28] Niklas Wretblad, Fredrik Storås, Martin Ingebrigtsen, Halvor Björklund, Sushant Nayak, and Petter Lundberg. 2024. Synthetic SQL Column Descriptions and Their Impact on Text-to-SQL Performance. *arXiv preprint arXiv:2408.04691* (2024).
- [29] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of EMNLP*.
- [30] Zhiqiang Zhang, Liqiang Wen, and Wen Zhao. 2024. A GAIL Fine-Tuned LLM Enhanced Framework for Low-Resource Knowledge Graph Question Answering. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 3300–3309. doi:10.1145/3627673.3679753
- [31] Chengshuai Zhao, Bowen Zheng, Haoming Zhang, et al. 2025. CyberBOT: Towards Reliable Cybersecurity Education via Ontology-Grounded Retrieval Augmented Generation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM.
- [32] Yuxiang Zhao, Moritz Blum, Sander Witteveen, Peter Meltzer, and Sajjan Koshy. 2025. AGENTiGraph: A Multi-Agent Knowledge Graph Framework for Interactive, Domain-Specific LLM Chatbots. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM. doi:10.1145/3746252.3761459